Building the SFS-OS Seed Agent — Nori

Stack: Python, SQLite, Claude API (via OpenRouter), SFS-OS | **Domain:** AI Infrastructure / Python Agent Development

What This Is

SFS-OS is a self-hosted Python multi-agent system that runs weekly business and personal life audits. The **seed agent (Nori)** is the orchestrator — it boots the system, routes data, dispatches 9 specialist sub-agents, writes briefs, and persists findings to a SQLite database via `db.py`. This playbook covers how the full pipeline was built: the agent architecture, the database layer, and how everything is wired together.

Architecture Overview

Layer	File	What it does
Orchestrator	`seed.py`	Nori — boots the system, routes data, dispatches agents, saves briefs
Database	`lib/db.py`	SQLite persistence — posture items + API usage metering
Config	`config/neo_gents.json`	Client profile, milestone tracking, persona selection
Persona	`prompts/personas/nori.txt`	Nori's system prompt — COO + Chief of Staff identity
Memory	`memory/nori-memory.md`	Persistent state between weekly runs
Data drop	`data/inbox/`	Drop any file here — AI classifier routes it automatically
Output	`briefs/`	Generated weekly markdown briefs (business, personal, security)
Event log	`logs/events.jsonl`	Structured event log for every agent run

Sub-Agent Roster

Agent	Track	What it analyzes	Data needed
revenue_agent	Business	Leads, CRM notes, revenue trends	leads.csv or crm_notes.txt
milestone_agent	Business	Progress toward current level goal, blockers	Always runs
team_agent	Business	Team capacity, hire decisions	Always runs
risk_agent	Business	Synthesizes all business findings into risk summary	Always runs
money_agent	Personal	Bank transactions, spending patterns	bank_MONTH.csv
time_agent	Personal	Calendar and task time audit	calendar.txt
growth_agent	Personal	Goals progress and learning	goals.txt
family_agent	Personal	Family context and priorities	Always runs (uses config)
ciso_agent	Security	Security posture scan — wired to db.py findings	Always runs on business track

Step 1 — Project structure

```
mkdir -p sfs-os/agents/business
mkdir -p sfs-os/agents/personal
mkdir -p sfs-os/agents/security
mkdir -p sfs-os/lib
mkdir -p sfs-os/prompts/personas
mkdir -p sfs-os/memory
mkdir -p sfs-os/data/inbox
mkdir -p sfs-os/data/business
mkdir -p sfs-os/data/personal
mkdir -p sfs-os/briefs
mkdir -p sfs-os/logs
cd sfs-os
python3 -m venv venv
source venv/bin/activate
pip install anthropic python-dotenv
```

Step 2 — Build the database layer (db.py)

Before the seed agent, build the persistence layer. Two tables: `posture_items` for security/IT findings that survive across runs, and `meter_events` for API usage tracking.

```
# lib/db.py — key functions

init_db()
# Creates both tables if they don't exist. Safe to call every run.
# Called at the top of seed.py main() so tables always exist before agents run.

upsert_posture_item(domain, agent, title, detail, severity, source_file, run_date)
# Inserts a finding. If an open item with the same domain+agent+title already
# exists, updates it instead of creating a duplicate. Prevents finding bloat.
# Severity values: critical | high | medium | low | info

resolve_posture_item(item_id, status)
# Marks a finding resolved. Status: closed | accepted | wont_fix

get_posture_summary()
# Returns counts by severity for all open items.
# Used by CISO agent dashboard to show: 3 critical, 1 high, etc.

log_meter_event(agent, model, prompt_tokens, completion_tokens, latency_ms, success)
# Logs every API call. Lets you track spend per agent per run.
```

Step 3 — Wire db.py into seed.py

The key connection: `init_db()` is called once at startup, before any agents run. This guarantees tables exist. The CISO agent then writes findings to `posture_items` via `upsert_posture_item()` during its run.

```
# In seed.py main():

from lib.db import init_db

def main():
    config = load_config()
    client = get_client()

    # Initialize database — creates tables if they don't exist
    init_db()

    # Now safe to run agents that write to the DB
    emit_event("plan", "active", config)
    ...
```

> **Why upsert instead of insert?** The CISO agent runs every week. Without upsert, you'd get duplicate rows for the same open finding (e.g., "SSH key rotation overdue") accumulating every run. Upsert checks for an existing open item with the same domain+agent+title — if found, it updates the detail and timestamp instead of creating a new row.

Step 4 — Build the seed agent orchestration flow

Nori's decision filter runs before dispatching agents — it checks what data is available and prints a readiness report:

```
def run_decision_filter(business_manifest, personal_manifest):
    has_biz = "YES" if has_data(business_manifest, "csv", "text") else "NO — add business data files"
    print(f"  1. Revenue support in 30 days?  {has_biz}")
    print(f"  2. Existing team member handles?  NO — Nori is orchestrating")
    print(f"  3. Coach Berry bottleneck?  CHECKING — milestone agent will assess")
    print(f"  4. Broken manual process?  CHECKING — risk agent will assess")
```

Agents are gated on data availability — some always run, some skip if no data is present:

```
# Revenue only runs if there's actual data to analyze
if has_data(manifest, DATA_TYPE_CSV, DATA_TYPE_TEXT):
    results.append(revenue_agent.run(manifest, client, config))
else:
    skipped.append("revenue (no leads.csv or crm_notes.txt)")

# Milestone always runs — works from config even with no data files
results.append(milestone_agent.run(manifest, client, config))
```

Step 5 — Persistent memory between runs

After every run, Nori writes a memory file that gets loaded at the top of the next run. This gives the agent continuity — it knows last week's MRR, milestone progress, and open risks without re-reading all the data.

```
# Memory is written at end of main()
update_memory(all_results, config, persona_id)

# Memory is read at start of main()
memory = read_memory(persona_id)
if memory:
    print(f"[nori/{persona_id}] Memory loaded from last session.")
```

Memory file location: `memory/nori-memory.md` — human-readable markdown, versioned with each run date.

Step 6 — The inbox classifier (auto-routing)

Drop any file into `data/inbox/` and the classifier figures out where it belongs. Bank statement? Routes to `data/personal/`. CRM export? Routes to `data/business/`. Uses Claude to classify by content, not filename.

```
# Called before scanning in main()
classify_inbox(BASE_DIR, client)

# After classification, scan picks up correctly routed files
biz_manifest = scan(str(_data_root(config) / "business"))
per_manifest  = scan(str(_data_root(config) / "personal"))
```

Step 7 — Run it

```
cd ~/Desktop/SFS-OS
source venv/bin/activate

# Run both tracks
python seed.py

# Run business only
python seed.py --mode business

# Run personal only
python seed.py --mode personal
```

Step 8 — Drop data and get a brief

```
# Business data
cp leads.csv  data/business/
cp crm_notes.txt  data/business/

# Personal data
cp bank_march.csv  data/personal/
cp calendar.txt  data/personal/
cp goals.txt  data/personal/

# Or just drop anything in inbox — Nori routes it
cp mystery_file.csv  data/inbox/

# Run
python seed.py
```

Briefs are saved to `briefs/YYYY-MM-DD-business.md`, `briefs/YYYY-MM-DD-personal.md`, and `briefs/YYYY-MM-DD-security.md`.

Verification

- `python seed.py` runs without errors and prints agent results
- `ls briefs/` shows dated brief files after the run
- `ls data/sfs_os.db` confirms the SQLite database was created
- `cat memory/nori-memory.md` shows the memory file was written
- `cat logs/events.jsonl` shows structured event log entries

Why This Matters for Cloud Engineering

- Demonstrates multi-agent orchestration architecture — coordinator pattern with gated sub-agents is a core pattern in production AI systems.
- SQLite persistence layer with upsert logic shows database design thinking — not just "store everything," but smart deduplication to keep data clean across recurring runs.
- Event logging to JSONL, metered API usage tracking, and structured memory files are the observability practices cloud engineers implement on real AI infrastructure.
- Runs on self-hosted OCI infrastructure at $0/month — proof of full-stack ownership from agent code to cloud deployment.

Full source: github.com/neogents-ai/GOER